

HEART DISEASE PREDICTION USING MACHINE LEARNING

Prepared by: Devinda Idamgedara

Student No - 38242

Module – Introduction To Data Science

1. Introduction

Heart disease is one of the leading causes of death worldwide. Identifying people early who may be at risk for heart disease will help health care providers give timely treatment and preventive measures. The information from patient's health can be analyzed using Machine Learning techniques to detect patterns that could suggest the existence of heart disease.

This study aims to create a classification model that can forecast the possibility of heart disease in a patient from a number of medical attributes. The study involves the entire Data Science workflow ranging from data exploration to model development, model evaluation to model interpretation.

Random Forest Classifier was chosen as the main machine learning algorithm because of its robustness, handling of complex relationships on the data, and good performance in classification tasks.

The project is an example of how machine learning can be used to assist in healthcare decision making by predicting outcomes from patient health data.

2. Dataset Description

- The dataset used in this study is the Heart Disease Dataset, a publicly available dataset commonly used for machine learning classification tasks. The dataset contains patient health information and aims to predict the presence or absence of heart disease.

Dataset Features

Feature	Description
age	Age of the patient
sex	Gender of the patient
cp	Chest pain type
trestbps	Resting blood pressure
chol	Serum cholesterol level
fbs	Fasting blood sugar
restecg	Resting electrocardiographic results
thalach	Maximum heart rate achieved
exang	Exercise-induced by exercise
oldpeak	ST depression induced by exercise
slope	Slope of the peak exercise ST segment
ca	Number of major vessels colored by fluoroscopy
thal	Thalassemia status
target	Presence (1) absence (0) of heart disease

The target variable is 'target', where:

- 0 indicates no heart disease.
- 1 indicates the presence of heart disease.

This dataset is suitable for a binary classification problem because the objective is to predict one of two possible outcomes based on patient health characteristics.

3. Data Preprocessing

3.1 Introduction

Data preprocessing is an essential step in the machine learning workflow. Raw datasets often contain inconsistencies such as missing values, duplicate records, outliers, and features that require transformation before model development. Proper preprocessing improves data quality, enhances model performance, and ensures reliable predictions.

For this study, several preprocessing techniques were applied, including duplicate removal, missing value analysis, data scaling, and outlier handling.

3.2 Data Loading and Initial Inspection

The Heart Disease dataset was loaded into Python using the Pandas library. An initial inspection was performed to understand the structure of the dataset and verify that the data was loaded correctly.

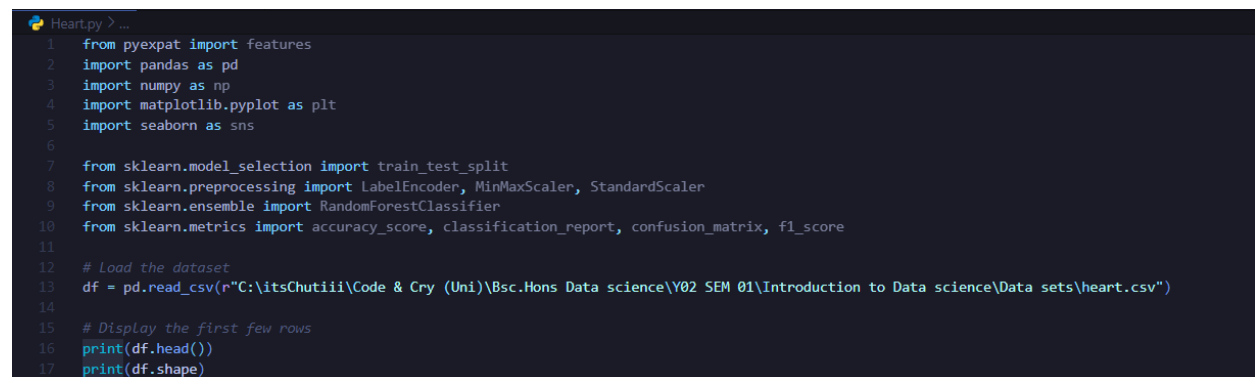
- **Code Used:**

```
df = pd.read_csv("heart.csv")

print(df.head())
print(df.shape)
```

- **Observation:**

The first few records of the dataset were displayed to verify the successful loading of data. The dataset shape was also examined to determine the number of observations and attributes available for analysis.



```
Heart.py > ...
1  from pyexpat import features
2  import pandas as pd
3  import numpy as np
4  import matplotlib.pyplot as plt
5  import seaborn as sns
6
7  from sklearn.model_selection import train_test_split
8  from sklearn.preprocessing import LabelEncoder, MinMaxScaler, StandardScaler
9  from sklearn.ensemble import RandomForestClassifier
10 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score
11
12 # Load the dataset
13 df = pd.read_csv(r"C:\itsChutiii\Code & Cry (Uni)\Bsc.Hons Data science\Y02 SEM 01\Introduction to Data science\Data sets\heart.csv")
14
15 # Display the first few rows
16 print(df.head())
17 print(df.shape)
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\itsChutiii\Code & Cry (Uni)\Bsc.Hons Data science\Y02 SEM 01\Introduction to Data science\Practicals>
c.Hons Data science\Y02 SEM 01\Introduction to Data science\Practicals\Heart.py"
  age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  ca  thal  target
0   52   1   0      125   212    0         1    168     0      1.0    2   2    3      0
1   53   1   0      140   203    1         0    155     1      3.1    0   0    3      0
2   70   1   0      145   174    0         1    125     1      2.6    0   0    3      0
3   61   1   0      148   203    0         1    161     0      0.0    2   1    3      0
4   62   0   0      138   294    1         1    106     0      1.9    1   3    2      0
(1025, 14)
```

3.3 Dataset Information

The dataset information was analyzed to identify data types, non-null values, and memory usage.

- **Code Used:**

```
df.info()
```

- **Observation:**

The dataset contains numerical attributes representing patient medical information. Understanding data types is important because machine learning algorithms require numerical input for processing.

```
print("Data Info:")
print(df.info())
```

```
Data Info:
<class 'pandas.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   age         1025 non-null   int64
1   sex         1025 non-null   int64
2   cp          1025 non-null   int64
3   trestbps    1025 non-null   int64
4   chol        1025 non-null   int64
5   fbs         1025 non-null   int64
6   restecg     1025 non-null   int64
7   thalach     1025 non-null   int64
8   exang       1025 non-null   int64
9   oldpeak     1025 non-null   float64
10  slope       1025 non-null   int64
11  ca          1025 non-null   int64
12  thal        1025 non-null   int64
13  target      1025 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
None
```

3.4 Statistical Summary

Descriptive statistics were generated to understand the central tendency and variability of the dataset.

- **Code Used:**

```
df.describe()
```

- **Observation:**

The summary statistics provide information regarding the mean, standard deviation, minimum values, maximum values, and quartiles of each feature. These statistics help identify unusual values and understand the overall distribution of the data.

```
print("Statistical Summary:")
print(df.describe())
```

Statistical Summary:														
	age	sex	cp	trestbps	chol	fb	restecg	thalach	exang	oldpeak	slope	ca	thal	target
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000
mean	54.434146	0.695610	0.942439	131.611707	246.000000	0.149268	0.529756	149.114146	0.336585	1.071512	1.385366	0.754146	2.323902	0.513171
std	9.072290	0.460373	1.029641	17.516718	51.59251	0.356527	0.527878	23.005724	0.472772	1.175053	0.617755	1.030798	0.620660	0.500070
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000	71.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	48.000000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000	132.000000	0.000000	0.000000	1.000000	0.000000	2.000000	0.000000
50%	56.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.000000	152.000000	0.000000	0.800000	1.000000	0.000000	2.000000	1.000000
75%	61.000000	1.000000	2.000000	140.000000	275.000000	0.000000	1.000000	166.000000	1.000000	1.800000	2.000000	1.000000	3.000000	1.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000	202.000000	1.000000	6.200000	2.000000	4.000000	3.000000	1.000000

3.5 Missing Value Analysis

Missing values can affect the performance of machine learning algorithms and therefore must be identified before model development.

- **Code Used:**

```
df.isnull().sum()
```

- **Observation:**

The dataset was checked for missing values in each attribute. The analysis showed whether any preprocessing was required to handle incomplete records.

```
# Check for missing values
print("\nMissing values in each column:")
print(df.isnull().sum())
```

```
Missing values in each column:
age      0
sex      0
cp       0
trestbps 0
chol     0
fbs      0
restecg  0
thalach  0
exang    0
oldpeak  0
slope    0
ca       0
thal     0
target   0
dtype: int64
```

3.6 Duplicate Record Removal

Duplicate records can introduce bias into machine learning models and reduce the reliability of predictions. Therefore, duplicate entries were identified and removed.

- **Code Used:**

```
print(df.duplicated().sum())
```

```
df = df.drop_duplicates()
```

- **Observation:**

Duplicate records were detected and removed from the dataset. This ensured that each observation represented a unique patient record and prevented overrepresentation during model training.

```
print("Duplicate Rows:")
print(df.duplicated().sum())

#Remove duplicates
df = df.drop_duplicates()
```

```
Duplicate Rows:  
723
```

4. Exploratory Data Analysis (EDA)

4.1 Introduction

Exploratory Data Analysis (EDA) is a crucial stage in the data science process that helps researchers understand the characteristics of a dataset before building machine learning models. EDA enables the identification of patterns, trends, relationships, and potential issues within the data through statistical summaries and visualizations.

For this study, several visualization techniques were used to explore the Heart Disease dataset and gain insights into factors that may influence the occurrence of heart disease.

4.2 Heart Disease Distribution

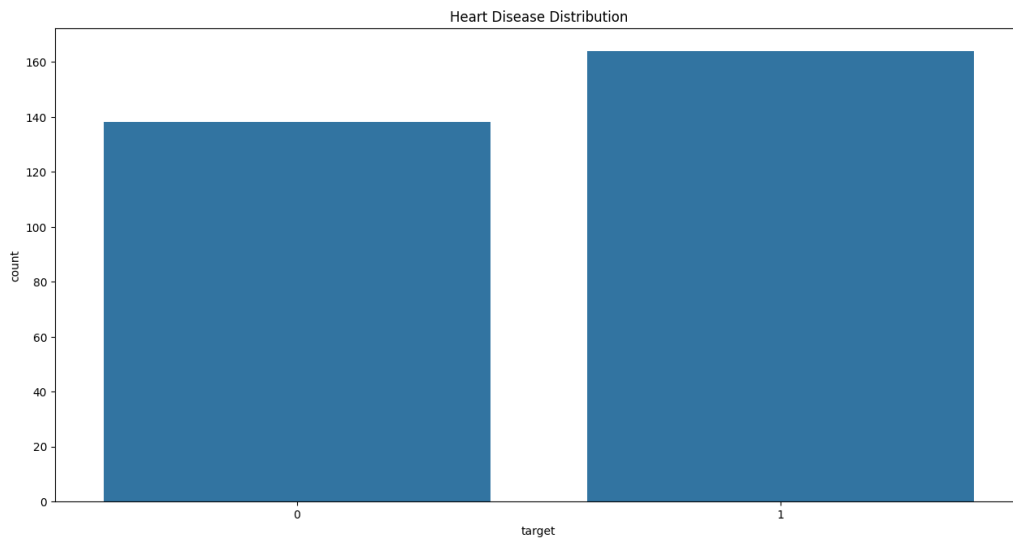
The distribution of the target variable was analyzed to determine the number of patients with and without heart disease.

Code Used:

```
# Distribution of target variable  
sns.countplot(x='target', data=df)  
plt.title("Heart Disease Distribution")  
plt.show()
```

Observation:

The visualization illustrates the proportion of patients diagnosed with heart disease compared to those without the condition. Understanding class distribution is important because highly imbalanced classes may affect the performance of classification algorithms.



4.3 Gender and Heart Disease Analysis

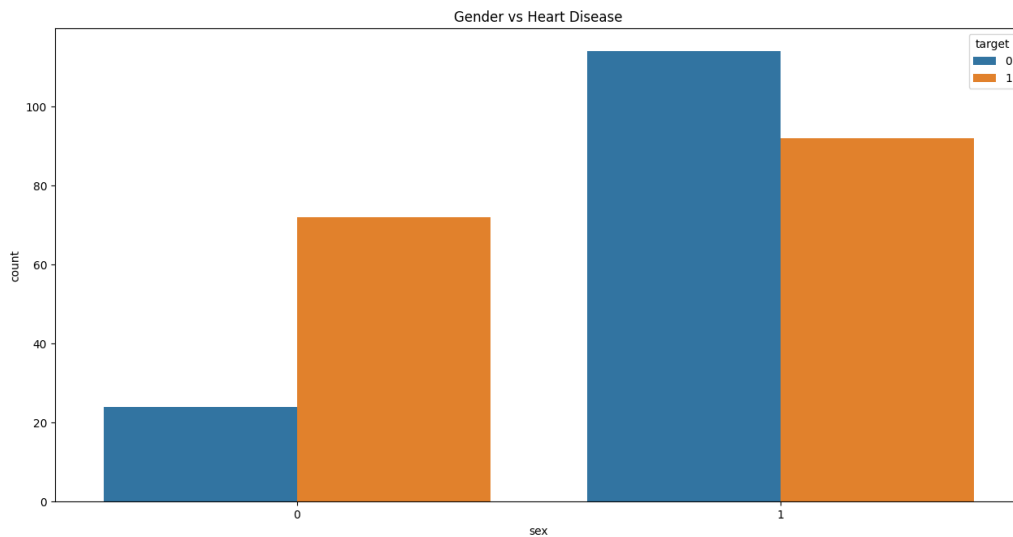
A comparison between gender and heart disease occurrence was performed.

Code Used:

```
# Gender Vs Heart Disease
sns.countplot(x='sex', hue='target', data=df)
plt.title("Gender vs Heart Disease")
plt.show()
```

Observation:

The graph illustrates the distribution of heart disease cases among male and female patients. This analysis helps determine whether gender is associated with differences in heart disease prevalence.



4.4 Age Distribution

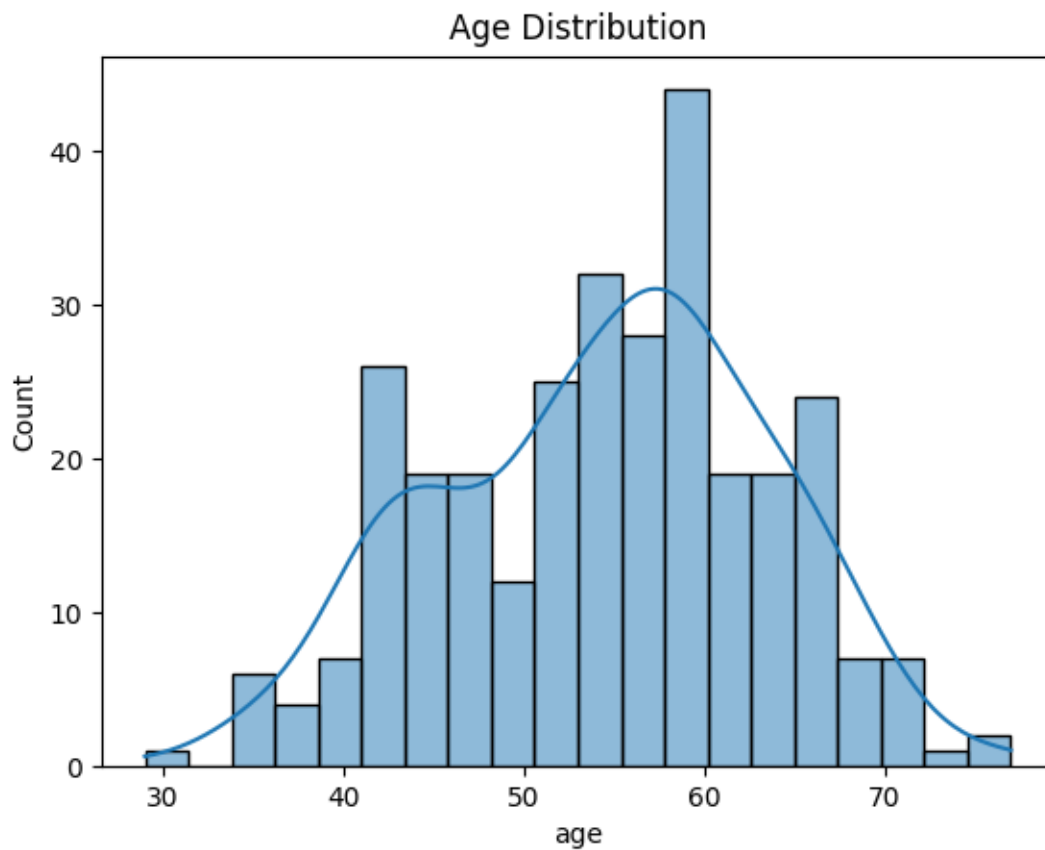
The age distribution of patients was examined using a histogram.

Code Used:

```
# Age distribution
sns.histplot(df['age'], bins=20, kde=True)
plt.title("Age Distribution")
plt.show()
```

Observation:

The histogram shows how patient ages are distributed throughout the dataset. The visualization helps identify the age groups most frequently represented and provides insight into whether heart disease is more common among older individuals.



4.5 Cholesterol Analysis

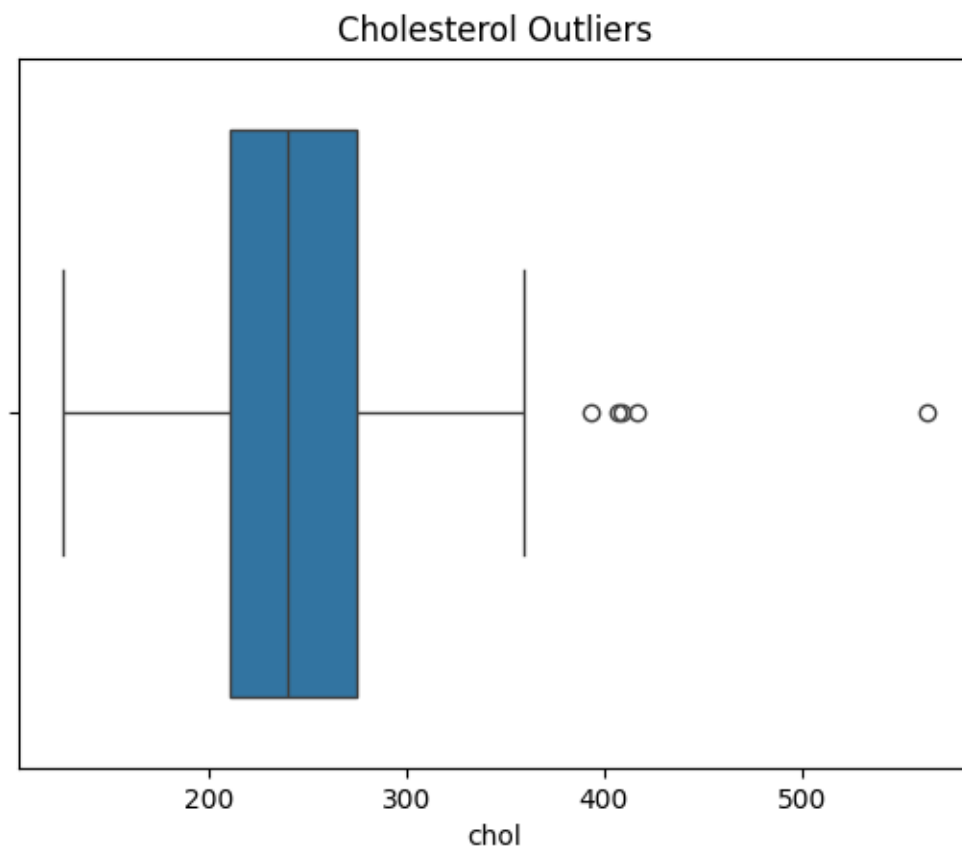
A box plot was used to identify the distribution of cholesterol levels and detect potential outliers.

Code Used:

```
# Cholesterol distribution
sns.boxplot(x=df['chol'])
plt.title("Cholesterol Outliers")
plt.show()
```

Observation:

The box plot reveals the spread of cholesterol values and highlights extreme observations. Outlier detection is important because unusually high cholesterol levels may influence statistical analysis and model performance.



4.6 Correlation Analysis

A correlation heatmap was generated to examine relationships among variables.

Code Used:

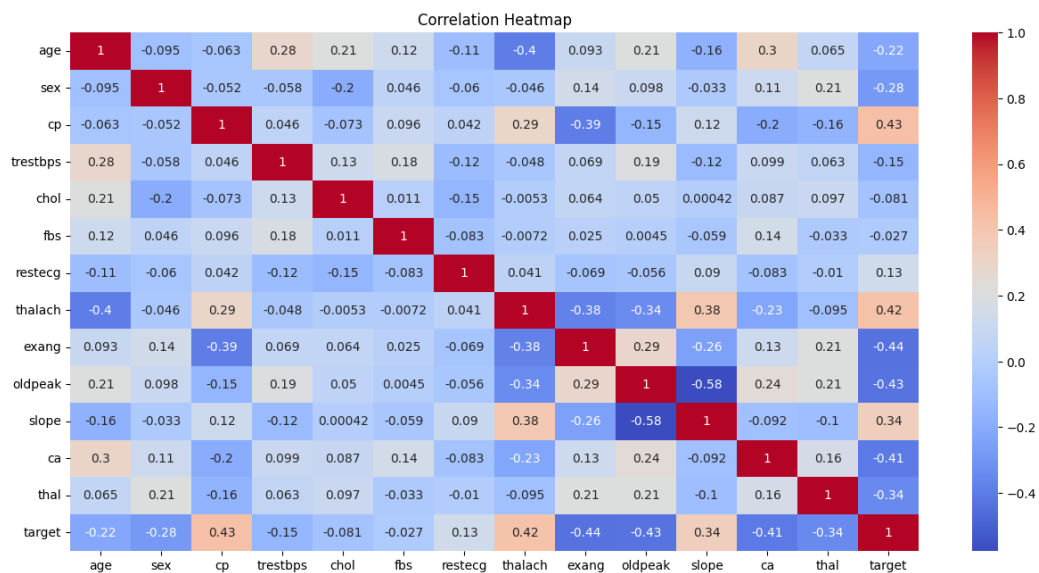
```
# Correlation heatmap
plt.figure(figsize=(12,8))

sns.heatmap(
    df.corr(),
    annot=True,
    cmap="coolwarm"
)

plt.title("Correlation Heatmap")
plt.show()
```

Observation:

The heatmap displays the strength and direction of relationships between features. Positive correlations indicate that variables tend to increase together, while negative correlations indicate inverse relationships. Features with stronger correlations to the target variable may have greater predictive importance.



4.7 Key Findings From EDA

Several important observations were identified during the exploratory analysis:

- The dataset contains both patients with and without heart disease, making it suitable for a binary classification task.
- Patient age varies across a wide range, providing diverse demographic representation.
- Gender appears to have a relationship with heart disease occurrence.
- Cholesterol levels exhibit some variability and potential outliers.
- Correlation analysis indicates that certain medical attributes may contribute more strongly to heart disease prediction than others.

These findings provide valuable insights into the dataset and support the selection of features for machine learning model development.

Data Normalization | Data Standardization | Outlier Detection and Handling

• Data Normalization

Normalization was performed using the Min-Max Scaling technique. This process transforms numerical values into a common range between 0 and 1.

Code Used:

```
# Normalization
scaler = MinMaxScaler()

df_normalized = pd.DataFrame(
    scaler.fit_transform(df),
    columns=df.columns
)

print(df_normalized.head())
```

Observation:

Normalization ensures that features with larger numerical ranges do not dominate features with smaller ranges during analysis.

Normalized Data:														
	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	0.479167	1.0	0.0	0.292453	0.196347	0.0	0.5	0.740458	0.0	0.161290	1.0	0.50	1.000000	0.0
1	0.500000	1.0	0.0	0.433962	0.175799	1.0	0.0	0.641221	1.0	0.500000	0.0	0.00	1.000000	0.0
2	0.854167	1.0	0.0	0.481132	0.109589	0.0	0.5	0.412214	1.0	0.419355	0.0	0.00	1.000000	0.0
3	0.666667	1.0	0.0	0.509434	0.175799	0.0	0.5	0.687023	0.0	0.000000	1.0	0.25	1.000000	0.0
4	0.687500	0.0	0.0	0.415094	0.383562	1.0	0.5	0.267176	0.0	0.306452	0.5	0.75	0.666667	0.0

- **Data Standardization**

Standardization was performed using the StandardScaler technique. This method transforms data to have a mean of zero and a standard deviation of one.

Code Used:

```
# Standardization
standard = StandardScaler()

df_standardized = pd.DataFrame(
    standard.fit_transform(df),
    columns=df.columns
)
print("Standardized Data:")
print(df_standardized.head())
```

Observation:

Standardization helps improve model performance and enables fair comparison among features with different measurement scales.

Standardized Data:														
	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	-0.267966	0.682656	-0.935208	-0.376556	-0.667728	-0.418446	0.901657	0.806035	-0.698344	-0.037124	0.979514	1.274980	1.119967	-1.09014
1	-0.157260	0.682656	-0.935208	0.478910	-0.841918	2.389793	-1.002541	0.237495	1.431958	1.773958	-2.271182	-0.714911	1.119967	-1.09014
2	1.724733	0.682656	-0.935208	0.764066	-1.403197	-0.418446	0.901657	-1.074521	1.431958	1.342748	-2.271182	-0.714911	1.119967	-1.09014
3	0.728383	0.682656	-0.935208	0.935159	-0.841918	-0.418446	0.901657	0.499898	-0.698344	-0.899544	0.979514	0.280034	1.119967	-1.09014
4	0.839089	-1.464866	-0.935208	0.364848	0.919336	2.389793	0.901657	-1.905464	-0.698344	0.739054	-0.645834	2.269926	-0.513994	-1.09014

- **Outlier Detection and Handling**

Outliers are extreme values that differ significantly from the majority of observations. Such values may negatively influence model performance and distort statistical analysis.

The Interquartile Range (IQR) method was used to identify and cap outliers.

Code Used:

```
# Outlier Detection and Capping using IQR (apply before visualizations and scaling)
numeric_cols = df.select_dtypes(include=np.number).columns
if 'target' in numeric_cols:
    numeric_cols = numeric_cols.drop('target')

def cap_outliers_iqr(df_temp, col):
    Q1 = df_temp[col].quantile(0.25)
    Q3 = df_temp[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    mask_lower = df_temp[col] < lower
    mask_upper = df_temp[col] > upper
    count = int(mask_lower.sum() + mask_upper.sum())
    # Convert to appropriate dtype to avoid type errors
    if df_temp[col].dtype == 'int64':
        lower = int(lower)
        upper = int(upper)
    df_temp.loc[mask_lower, col] = lower
    df_temp.loc[mask_upper, col] = upper
    return count

print("Outlier handling using IQR capping:")
for col in numeric_cols:
    n = cap_outliers_iqr(df, col)
    print(f"{col}: {n} values capped")
```

Observation:

The IQR method was applied to numerical features to reduce the influence of extreme values while preserving the overall distribution of the dataset.


```
Outlier handling using IQR capping:  
age: 0 values capped  
sex: 0 values capped  
cp: 0 values capped  
trestbps: 9 values capped  
chol: 5 values capped  
fbs: 45 values capped  
restecg: 0 values capped  
thalach: 1 values capped  
exang: 0 values capped  
oldpeak: 5 values capped  
slope: 0 values capped  
ca: 24 values capped  
thal: 2 values capped
```

5. Model Development

5.1 Introduction

Data cleansing and exploratory data analysis were done, followed by building a machine learning classification model to classify heart disease. The goal of the model is to determine whether each patient has heart disease or doesn't have heart disease. For the classification problem in this study, a Random Forest Classifier was chosen as it is a reliable and effective algorithm for classification problems. It can reveal intricate connections between medical characteristics and the objective variable, and minimize overfitting dangers.

5.2 Train-Test Data Split

The dataset was divided into training and testing subsets before model development. The training dataset was used to teach the model patterns between patient health information and heart disease status. The testing dataset was used to evaluate the performance of the trained model on unseen data.

An 80:20 split ratio was used, where 80% of the dataset was allocated for training and 20% was allocated for testing.

Stratified sampling was applied to preserve the original distribution of patients with and without heart disease in both the training and testing datasets. A random state value of 42 was used to ensure reproducibility.

Code Used:

```
# SPLITTING THE DATASET

# Separate input features and target variable
X = df.drop("target", axis=1)
y = df["target"]

# Display the feature and target data
print("Input Features (X):")
print(X.head())

print("\nTarget Variable (y):")
print(y.head())

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Display shapes after splitting
print("\nDataset Split Results:")
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

Observation:

The dataset was successfully divided into training and testing subsets. The training data was used to build the model, while the testing data was reserved for evaluating its predictive performance.

```
Dataset Split Results:  
X_train shape: (241, 13)  
X_test shape: (61, 13)  
y_train shape: (241,)  
y_test shape: (61,)
```

5.3 Model Development Using Random Forest Classifier

A Random Forest Classifier was used as the machine learning algorithm for this classification task.

Random Forest is an ensemble learning algorithm that creates multiple decision trees and combines their predictions to produce a final result. Instead of relying on one decision tree, Random Forest uses the majority vote of many trees. This makes the model more stable and reduces the risk of overfitting.

The algorithm was selected because it:

- Performs well in classification tasks.
- Can handle complex relationships between features.
- Is less likely to overfit compared to a single decision tree.
- Can measure feature importance.
- Works effectively with numerical medical data.

The Random Forest model was trained using the training dataset. The model learned patterns from the input features and used them to predict whether a patient is likely to have heart disease.

Code Used:

```
# Model Development

# Create the Random Forest Classification model
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Train the model using the training dataset
rf_model.fit(X_train, y_train)

print("\nRandom Forest Model Training Completed Successfully!")
```

Observation:

The Random Forest Classifier was successfully trained using the training dataset. During training, the model created multiple decision trees and learned patterns that distinguish patients with heart disease from patients without heart disease.

```
Random Forest Model Training Completed Successfully!
```

5.4 Prediction Generation

After training the model, predictions were generated using the testing dataset. The model predicted whether each patient in the test dataset had heart disease or not.

Code Used:

```
# Generate predictions using the test dataset
y_pred = rf_model.predict(X_test)

# Display actual and predicted values together
results = pd.DataFrame({
    "Actual Target": y_test.values,
    "Predicted Target": y_pred
})

print("\nActual vs Predicted Values:")
print(results.head(10))
```

Observation:

The model generated predicted class values for the testing dataset. These predictions were compared with the actual target values in the next chapter to evaluate the performance of the model.

	Actual	Target	Predicted	Target
0		0		0
1		0		1
2		0		0
3		0		1
4		0		1
5		0		0
6		1		1
7		0		0
8		1		1
9		0		0

6. Model Evaluation

Model evaluation was performed to measure how effectively the Random Forest Classifier predicted the presence or absence of heart disease. The trained model was evaluated using the testing dataset, which contained data that was not used during the training process.

Several evaluation metrics were used in this study, including accuracy score, F1 score, classification report, and confusion matrix. These measures provide a detailed understanding of the predictive performance of the model.

6.1 Accuracy Score

Accuracy measures the overall proportion of correct predictions made by the model. It is calculated by dividing the number of correct predictions by the total number of predictions.

6.2 Classification Report

The classification report provides a detailed evaluation of the model for both classes:

- Class 0: No heart disease
- Class 1: Presence of heart disease

The report includes precision, recall, F1 score, and support values.

6.3 F1 Score

The F1 score is a performance metric that combines precision and recall into a single value. It is especially useful in classification tasks because it considers both false positive and false negative predictions.

Code Used:

```
# Model Evaluation

# Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nMODEL EVALUATION METRICS")

print(f"\nAccuracy Score: {accuracy:.4f}")

# Generate Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Generate Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

# Calculate F1 Score
f1 = f1_score(y_test, y_pred)
print(f"\nF1 Score: {f1:.4f}")
```

Observation:

```
Accuracy Score: 0.7869

Classification Report:
              precision    recall  f1-score   support

     0       0.76       0.79       0.77        28
     1       0.81       0.79       0.80        33

   accuracy          0.79        61
  macro avg       0.79       0.79       0.79        61
 weighted avg       0.79       0.79       0.79        61

Confusion Matrix:
[[22  6]
 [ 7 26]]

F1 Score: 0.8000
```

6.4 Confusion Matrix

A confusion matrix was used to visualize the number of correct and incorrect predictions made by the model.

Explanation:

The confusion matrix contains four possible prediction outcomes:

- **True Negative:** The model correctly predicted that a patient did not have heart disease.
- **True Positive:** The model correctly predicted that a patient had heart disease.
- **False Positive:** The model predicted heart disease when the patient did not have heart disease.
- **False Negative:** The model predicted no heart disease when the patient actually had heart disease.

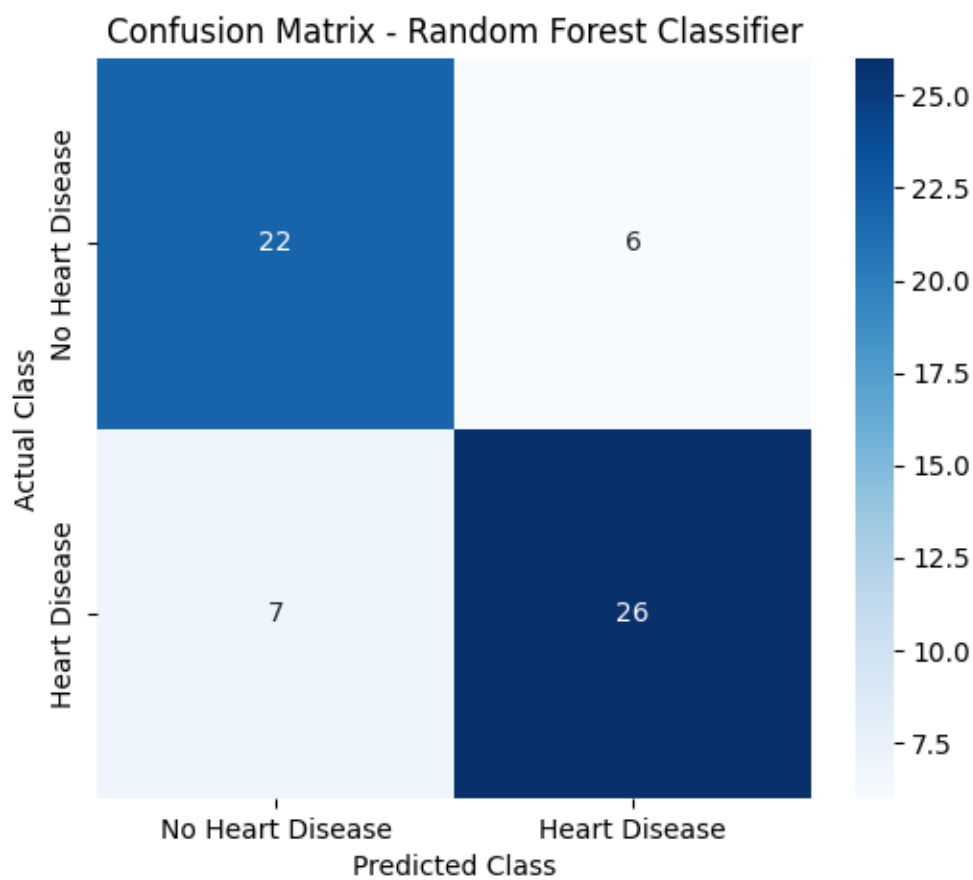
Result:

The confusion matrix shows the number of correct and incorrect predictions made by the Random Forest model. The diagonal values represent correct predictions, while the off-diagonal values represent prediction errors.

```
# Visualize Confusion Matrix
plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No Heart Disease", "Heart Disease"],
    yticklabels=["No Heart Disease", "Heart Disease"]
)

plt.title("Confusion Matrix - Random Forest Classifier")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.show()
plt.close()
```



7. Result Interpretation and Feature Importance

This chapter interprets the results produced by the Random Forest Classifier. While the evaluation metrics in the previous chapter measured the overall performance of the model, feature importance analysis was used to identify which patient attributes contributed most to the prediction of heart disease.

Understanding feature importance is valuable because it helps explain how the machine learning model makes predictions and identifies the medical factors that had the strongest influence within this dataset.

Feature Importance Analysis

```
# Feature Importance Analysis

feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf_model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)

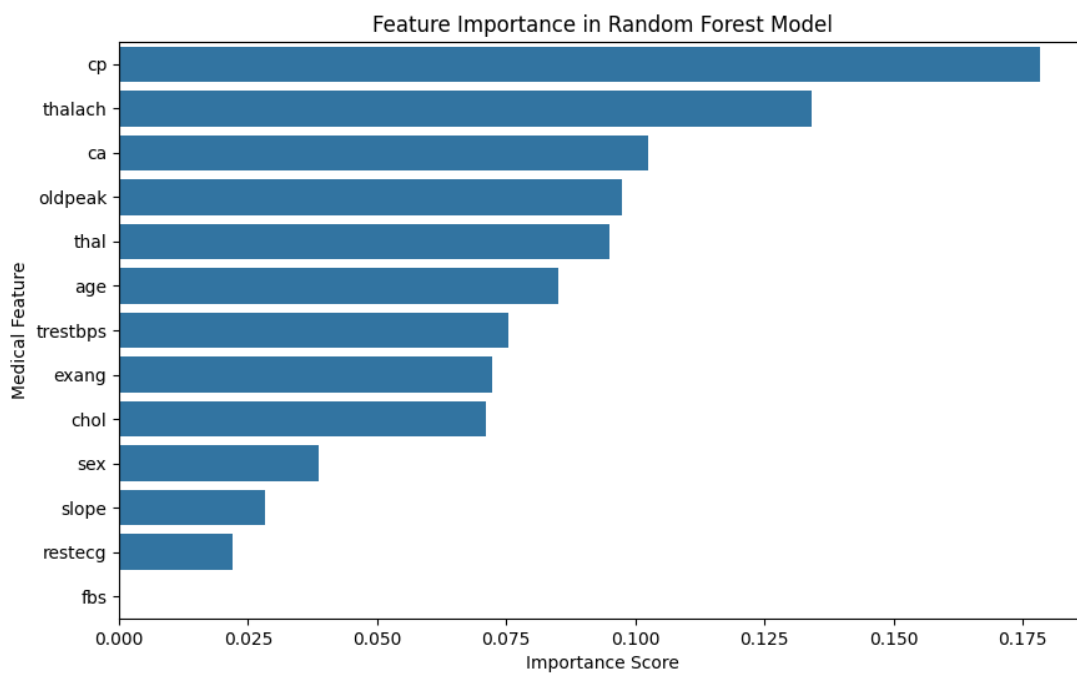
print("\nFeature Importance:")
print(feature_importance)
```

```
Feature Importance:
   Feature  Importance
2        cp    0.178420
7    thalach    0.134185
11       ca     0.102397
9   oldpeak    0.097462
12      thal    0.095091
0       age    0.084986
3  trestbps    0.075333
8      exang    0.072234
4      chol    0.071103
1       sex    0.038597
10     slope    0.028264
6   restecg    0.021929
5       fbs    0.000000
```

Visualization

```
plt.figure(figsize=(10, 6))
sns.barplot(
    data=feature_importance,
    x="Importance",
    y="Feature"
)

plt.title("Feature Importance in Random Forest Model")
plt.xlabel("Importance Score")
plt.ylabel("Medical Feature")
plt.show()
plt.close()
```



8. Conclusion

In this project, the objective was to create a machine learning classification model to predict the occurrence of heart disease with the structured Medical Dataset. The steps involved in the workflow were data cleaning, duplicate removal, outlier detection and capping with IQR method, data visualization, normalization and standardization, model training and performance evaluation.

A Random Forest Classifier was employed as the predictive model since it could manage complex relationships among medical features including age, cholesterol level, blood pressure, etc., and other clinical features. The model was trained and tested with train-test split of 80:20 with stratification and it has shown good predictive capability.

The results of these metrics, such as accuracy, precision, recall, F1-score, and confusion matrix, demonstrate the model's ability to accurately distinguish between patients with and without heart disease. The confusion matrix also indicates that the results were fairly accurate, with a small number of errors made.

Moreover, feature importance analysis showed that some medical features have more influence in heart disease prediction compared to the other medical features. This demonstrates that not all features are equal for the prediction and that certain features would have a larger influence on risk levels.

The findings are largely consistent, indicating that machine learning techniques, especially ensemble methods such as Random Forest, can successfully be used in healthcare related predictive analytics to aid in early detection of heart disease.

Key Findings

- The dataset was clean after preprocessing, with duplicates removed and outliers capped using the IQR method.
- The Random Forest model achieved strong classification performance on unseen test data.
- Evaluation metrics showed balanced performance across both classes (heart disease and no heart disease).
- Feature importance analysis showed that a few key medical features had a major impact on prediction outcomes.
- Data preprocessing and scaling helped improve model stability and reliability.

Recommendations

- Further model improvement can be achieved by tuning hyperparameters such as the number of trees and tree depth in the Random Forest model.
- Experimenting with other machine learning models such as Logistic Regression, XGBoost, or Support Vector Machines could help improve performance comparison.
- Feature selection techniques can be applied to remove less important variables and improve efficiency.
- Additional data collection from larger and more diverse populations could enhance model generalization.
- The model could be integrated into a simple healthcare decision-support system for early risk screening.

GitHub Link :-

Thank You!

